

Assignment

Joins & its Types

Hands-on Scenario-Based Practice

Q1. Basic INNER JOIN

Write a query that lists every student's full_name together with their department name. Use an INNER JOIN. Order by dept_name, then full_name.

SQL QUERY-

```
80 SELECT
81     s.student_name,
82     d.dept_name
83 FROM academics.students s
84 INNER JOIN academics.departments d
85     ON s.dept_id = d.dept_id
86 ORDER BY d.dept_name, s.student_name;
```

Data Output Messages Notifications

	student_name character varying (100)	dept_name character varying (100)
1	Priya Verma	Computer Science
2	Reetiks	Computer Science
3	Sunakshi	Computer Science
4	Test Student	Computer Science
5	Vanshika	Computer Science

Q2. LEFT JOIN — Students without a Department

Some students may still have NULL dept_id. Write a LEFT JOIN query that returns all students and their department name (NULL if unassigned). Then add a WHERE clause to show only the students who have no department assigned.

SQL QUERY-

a. All students with department name

```
89 SELECT
90     s.student_id,
91     s.student_name,
92     d.dept_name
93 FROM academics.students s
94 LEFT JOIN academics.departments d
95     ON s.dept_id = d.dept_id;
```

Data Output Messages Notifications

	student_id integer	student_name character varying (100)	dept_name character varying (100)
1	9999	Test Student	Computer Science
2	1	Sunakshi	Computer Science
3	4	Vanshika	Computer Science
4	2	Reetiks	Computer Science
5	3	Priya Verma	Computer Science
6	2001	Rahul Sharma	[null]

b. Only students with no department

```

98  SELECT
99      s.student_id,
100     s.student_name,
101     d.dept_name
102  FROM academics.students s
103  LEFT JOIN academics.departments d
104        ON s.dept_id = d.dept_id
105  WHERE d.dept_id IS NULL;

```

Data Output				Messages	Notifications
	student_id integer	student_name character varying (100)	dept_name character varying (100)		
1	2001	Rahul Sharma	[null]		

Q3. Multi-table INNER JOIN

Produce a report showing: student full_name, course_name, grade, and department name of the student.

Join students → enrollments → courses → departments. Only include graded enrollments (grade IS NOT NULL). Order by full_name, course_name.

SQL QUERY-

```
139 SELECT
140     s.student_name,
141     c.course_name,
142     e.grade,
143     d.dept_name
144 FROM academics.students s
145 INNER JOIN academics.enrollments e
146     ON s.student_id = e.student_id
147 INNER JOIN academics.courses c
148     ON e.course_id = c.course_id
149 INNER JOIN academics.departments d
150     ON s.dept_id = d.dept_id
151 WHERE e.grade IS NOT NULL
152 ORDER BY s.student_name, c.course_name;
```

Data Output

Messages

Notifications

SQL

	student_name character varying (100)	course_name character varying (100)	grade character (2)	dept_name character varying (100)
1	Priya Verma	Database	B	Computer Science
2	Priya Verma	Database Management Syste...	B+	Computer Science
3	Priya Verma	Operating Systems	A	Computer Science
4	Reetiks	Computer Networks	B	Computer Science
5	Reetiks	Database	A	Computer Science
6	Reetiks	Database Management Syste...	A+	Computer Science
7	Sunakshi	Computer Networks	B+	Computer Science
8	Sunakshi	Database Management Syste...	A	Computer Science
9	Sunakshi	Operating Systems	A	Computer Science
10	Vanshika	Computer Networks	B+	Computer Science
11	Vanshika	Database Management Syste...	A	Computer Science
12	Vanshika	Operating Systems	A+	Computer Science

Q4. Find Students Not Enrolled in Any Course

The college wants a list of students who have not enrolled in even a single course. Write a LEFT JOIN (anti-join pattern) that returns student_id and full_name of such students.

SQL QUERY-

```
155 SELECT
156     s.student_id,
157     s.student_name
158 FROM academics.students s
159 LEFT JOIN academics.enrollments e
160     ON s.student_id = e.student_id
161 WHERE e.student_id IS NULL;
162
```

Data Output			Messages	Notifications
	student_id [PK] integer	student_name character varying (100)		
1	9999	Test Student		
2	2001	Rahul Sharma		

Q5. Courses Never Taken

Similarly, list all courses that have zero enrollments (course_id and course_name). Use LEFT JOIN + IS NULL pattern.

SQL QUERY-

```
164 SELECT
165     c.course_id,
166     c.course_name
167 FROM academics.courses c
168 LEFT JOIN academics.enrollments e
169     ON c.course_id = e.course_id
170 WHERE e.course_id IS NULL;
171
```

Data Output			Messages	Notifications
	course_id [PK] integer	course_name character varying (100)		
1	101	Database		

Q6. Department-wise Student Count

Write a query that shows every department (even those with zero students) along with the count of students belonging to it. Use LEFT JOIN + COUNT + GROUP BY. Order by student count descending.

SQL QUERY-

```
175 SELECT
176     d.dept_id,
177     d.dept_name,
178     COUNT(s.student_id) AS student_count
179 FROM academics.departments d
180 LEFT JOIN academics.students s
181     ON d.dept_id = s.dept_id
182 GROUP BY d.dept_id, d.dept_name
183 ORDER BY student_count DESC;
```

Data Output			Messages	Notifications
	dept_id [PK] integer	dept_name character varying (100)	student_count bigint	
1	1	Computer Science	5	
2	4	Civil Engineering	0	
3	2	Mechanical Engineering	0	
4	3	Electronics Engineering	0	

Q7. RIGHT JOIN Practice

Using a RIGHT JOIN, list every course along with the names of students enrolled in it. Courses with no enrollments should still appear (student name will be NULL).

SQL QUERY-

```
187 SELECT
188     c.course_id,
189     c.course_name,
190     s.student_name
191 FROM academics.students s
192 RIGHT JOIN academics.enrollments e
193     ON s.student_id = e.student_id
194 RIGHT JOIN academics.courses c
195     ON e.course_id = c.course_id
196 ORDER BY c.course_name, s.student_name;
```

Data Output			Messages	Notifications
	course_id integer	course_name character varying (100)	student_name character varying (100)	
1	3	Computer Networks	Reetiks	
2	3	Computer Networks	Sunakshi	
3	3	Computer Networks	Vanshika	
4	100	Database	Priya Verma	
5	100	Database	Reetiks	
6	101	Database	[null]	
7	2	Database Management Syste...	Priya Verma	
8	2	Database Management Syste...	Reetiks	
9	2	Database Management Syste...	Sunakshi	
10	2	Database Management Syste...	Vanshika	
11	4	Operating Systems	Priya Verma	
12	4	Operating Systems	Sunakshi	
13	4	Operating Systems	Vanshika	

Q8. FULL OUTER JOIN — Orphans on Both Sides

Write a FULL OUTER JOIN between students and enrollments that reveals:

- Students with no enrollments, and
- Enrollments that somehow reference a non-existent student (should not happen if FKs are correct, but useful for data-quality checks).

Return student_id, full_name, course_id, grade. Filter to rows where either side is missing.

SQL QUERY-

```
199 SELECT
200     s.student_id,
201     s.student_name,
202     e.course_id,
203     e.grade
204 FROM academics.students s
205 FULL OUTER JOIN academics.enrollments e
206     ON s.student_id = e.student_id
207 WHERE s.student_id IS NULL
208     OR e.student_id IS NULL;
```

	student_id integer	student_name character varying (100)	course_id integer	grade character (2)
1	9999	Test Student	[null]	[null]
2	2001	Rahul Sharma	[null]	[null]

Q9. SELF JOIN — Employee Hierarchy

Using the employees table from Day 3 Q12 (or create it if missing: emp_id, emp_name, manager_id), write a self-join that displays each employee next to their manager's name. Include employees who have no manager (top-level). Use LEFT JOIN.

SQL QUERY-

```
222 SELECT
223     e.emp_id,
224     e.emp_name AS employee_name,
225     m.emp_name AS manager_name
226 FROM academics.employees e
227 LEFT JOIN academics.employees m
228     ON e.manager_id = m.emp_id
229 ORDER BY e.emp_id;
```

	emp_id [PK] integer	employee_name character varying (100)	manager_name character varying (100)
1	1	Dr. Principal	[null]
2	2	Dr. Sharma	Dr. Principal
3	3	Dr. Kapoor	Dr. Sharma
4	10	Raj	[null]
5	20	Aman	Dr. Principal
6	30	Priya	Dr. Principal
7	40	Neha	Dr. Sharma
8	50	Karan	Dr. Sharma
9	60	Simran	Dr. Kapoor

Q10. Students and Their Course Load

For each student, show: full_name, number of courses enrolled, and average grade (treat grade as text for now — just COUNT the graded enrollments). Include students with zero enrollments (show 0). Hint: LEFT JOIN + GROUP BY + COUNT.

SQL QUERY-

```
232 SELECT
233     s.student_name,
234     COUNT(e.course_id) AS course_count,
235     COUNT(e.grade) AS graded_enrollment_count
236 FROM academics.students s
237 LEFT JOIN academics.enrollments e
238     ON s.student_id = e.student_id
239 GROUP BY s.student_id, s.student_name
240 ORDER BY s.student_name;
```

	student_name character varying (100)	course_count bigint	graded_enrollment_count bigint
1	Priya Verma	3	3
2	Rahul Sharma	0	0
3	Reetiks	3	3
4	Sunakshi	3	3
5	Test Student	0	0
6	Vanshika	3	3

Q11. JOIN ... USING vs ON

- (a) Rewrite the query from Q1 using JOIN ... USING (dept_id) instead of ON.
- (b) In 1-2 sentences explain when USING is convenient and when it can become risky.

(a) Statement: Using USING

```
242 SELECT
243     s.student_name,
244     d.dept_name
245 FROM academics.students s
246 INNER JOIN academics.departments d
247     USING (dept_id)
248 ORDER BY d.dept_name, s.student_name;
```

	student_name character varying (100)	dept_name character varying (100)
1	Priya Verma	Computer Science
2	Reetiks	Computer Science
3	Sunakshi	Computer Science
4	Test Student	Computer Science
5	Vanshika	Computer Science

(b) Explanation: USING is convenient when both tables have the **same column name** for the join key. It can become risky when column names differ or when you need more complex join conditions, where ON is clearer and more flexible.

Q12. Complex Reporting Query

Produce a single result set that answers: "Which departments have students enrolled in courses that belong to a different department?"; (cross-department enrollments). Show: student_name, student_dept, course_name, course_dept. Only include rows where the two department names differ.

SQL QUERY-

```

251 SELECT
252     s.student_name AS student_name,
253     sd.dept_name AS student_dept,
254     c.course_name,
255     cd.dept_name AS course_dept
256 FROM academics.students s
257 INNER JOIN academics.departments sd
258     ON s.dept_id = sd.dept_id
259 INNER JOIN academics.enrollments e
260     ON s.student_id = e.student_id
261 INNER JOIN academics.courses c
262     ON e.course_id = c.course_id
263 INNER JOIN academics.departments cd
264     ON c.dept_id = cd.dept_id
265 WHERE sd.dept_name <> cd.dept_name
266 ORDER BY s.student_name, c.course_name;

```

Data Output Messages Notifications

	student_name character varying (100)	student_dept character varying (100)	course_name character varying (100)	course_dept character varying (100)
1	Priya Verma	Computer Science	Database	Mechanical Engineering
2	Priya Verma	Computer Science	Operating Systems	Mechanical Engineering
3	Reetiks	Computer Science	Computer Networks	Mechanical Engineering
4	Reetiks	Computer Science	Database	Mechanical Engineering
5	Sunakshi	Computer Science	Computer Networks	Mechanical Engineering
6	Sunakshi	Computer Science	Operating Systems	Mechanical Engineering
7	Vanshika	Computer Science	Computer Networks	Mechanical Engineering
8	Vanshika	Computer Science	Operating Systems	Mechanical Engineering